OpenTelemetry Implementation & Observability Report

AI-Based Village Pond & Catchment Planning System (HydroPond AI)

Course: Distributed Systems / Software Engineering Lab

Student Name: Karthik Vishlavath | **Roll / Email:** vishlavathk@iitbhilai.ac.in

Institution: Indian Institute of Technology Bhilai

Deployment Host / Container: `http://10.1.75.51:3317/`

Source Repository: https://github.com/VishlavthKarthik/Pond-Catchment-Plan

Date: October 16, 2026

Executive Summary

This report documents the end-to-end integration, deployment, and verification of **OpenTelemetry (OTel)** into the **AI-based Village Pond & Catchment Planning System (HydroPond AI)** from Assignment 1, deployed at `http://10.1.75.51:3317/`.

Modern backend systems require comprehensive observability beyond unstructured, isolated log messages. Using OpenTelemetry's vendor-neutral API and SDK, we instrumented all critical backend micro-stages: HTTP ingress routing, KML contour ingestion, Digital Elevation Model (DEM) rasterization, Wang-Liu priority-flood depression filling, D8 flow routing, upstream watershed delineation, and Central Water Commission (CWC) rational runoff calculations.

Observability data is continuously collected across the **Three Pillars of Observability**:

- **Distributed Tracing:** Exported via OpenTelemetry Protocol (OTLP/HTTP) to a **Jaeger** distributed tracing engine (`http://10.1.75.51:16686`), capturing end-to-end span waterfalls, call trees, and custom domain attributes.
- **Application & Infrastructure Metrics:** Aggregated via the OpenTelemetry `MeterProvider` and scraped at `http://10.1.75.51:3317/metrics` by a **Prometheus** Time-Series Database (`http://10.1.75.51:9090`), enabling real-time PromQL monitoring and capacity profiling.
- **Context-Correlated Structured Logs:** Embedded with W3C standard 128-bit `trace_id` and 64-bit `span_id` to allow instant bi-directional correlation between terminal server logs and Jaeger traces.

1. Description of the OpenTelemetry Implementation

1.1 Architectural Overview

HydroPond AI is an asynchronous GIS and hydrological decision-support platform designed to delineate catchment watersheds and calculate harvestable rainwater yield for rural village storage ponds. The computational pipeline processes complex geospatial vector data and high-resolution raster matrices.

Without instrumentation, this pipeline operates as a black box: slow requests cannot be attributed to specific algorithms, transient external network delays (e.g., historical climate API queries) remain invisible, and memory allocation spikes cannot be correlated with grid cell dimensions.



1.2 Telemetry Subsystem Implementation Details

The implementation resides in a modular telemetry subsystem (`app/telemetry.py`), ensuring clean separation of concerns and zero tight coupling with business logic:

1. **Tracer Provider (`TracerProvider`):** Configured with a standardized `Resource` descriptor identifying the service name (`hydrology-pond-system`), semantic version (`2.0.0`), and deployment environment. Attaches a `BatchSpanProcessor` coupled to `OTLPSpanExporter(endpoint="http://10.1.75.51:4318/v1/traces")` to asynchronously dispatch spans to the Jaeger collector without blocking the main event loop.
2. **Custom Span Hierarchy:** Every algorithmic phase is instrumented with explicit child spans (`tracer.start_as_current_span`), creating a rich call tree: `api.analyze_contour`, `kml.parse`, `dem.build`, `terrain.priority_flood_fill`, `pond.select_and_delineate`, `hydrology.calculate`, and `hydrology.fetch_historical_rainfall`.
3. **Domain-Specific Semantic Attributes:** Custom spans record critical hydrological attributes: `hydrology.annual_rainfall_mm`, `hydrology.runoff_coefficient`, `hydrology.runoff_volume_m3`, `catchment.area_hectares`, `catchment.mean_slope_pct`, and `dem.cell_count`.
4. **Metrics Subsystem:** Registers a `PrometheusMetricReader` providing a native Prometheus pull endpoint at `GET http://10.1.75.51:3317/metrics` with custom instruments: `hydrology_requests_total` (Counter), `hydrology_algorithm_duration_seconds` (Histogram), `hydrology_dem_grid_cells_total` (Histogram), and `hydrology_active_requests` (Gauge).
5. **Trace-Context Correlated Structured Logging:** Implemented a custom `TraceContextFormatter` subclassing Python's standard `logging.Formatter`. Automatically inspects `trace.get_current_span().get_span_context()` to inject `trace_id` and `span_id` into all log records, enabling instant bi-directional correlation between log lines and Jaeger traces.

2. Tools, Libraries, and Backend Services Used

Component	Technology	Version	Purpose in Project
Telemetry SDK	`opentelemetry-api`	`1.45.1`	Core OpenTelemetry interfaces, trace/metric context propagation, and span lifecycle.
Telemetry Runtime	`opentelemetry-sdk`	`1.45.1`	TracerProvider, MeterProvider, batch processors, and resource definitions.
FastAPI Instrumentation	`opentelemetry-instrumentation-fastapi`	`0.66b1`	Automatic HTTP request/response tracing, route identification, and status code tagging.
HTTP Client Instrumentation	`opentelemetry-instrumentation-httpx`	`0.66b1`	Distributed context propagation and span creation for external climate API requests.
Logging Instrumentation	`opentelemetry-instrumentation-logging`	`0.66b1`	Integration between standard Python `logging` and OpenTelemetry context.
OTLP Exporter	`opentelemetry-exporter-otlp-proto-http`	`1.45.1`	Protocol Buffers over HTTP exporter dispatching traces to Jaeger at `:4318/v1/traces`.
Prometheus Exporter	`opentelemetry-exporter-prometheus`	`0.66b1`	Bridges OpenTelemetry meter instruments into Prometheus format at `/metrics`.
Prometheus Client	`prometheus_client`	`0.26.0`	Official client generating metric exposition format and registry storage.
Tracing Backend	**Jaeger All-in-One**	`v1.60.0`	Standalone backend collecting OTLP spans, indexing traces, and UI at `http://10.1.75.51:16686`.
Metrics TSDB Backend	**Prometheus Server**	`v3.15.0`	Standalone TSDB scraping `http://10.1.75.51:3317/metrics` with UI at `http://10.1.75.51:9090`.
Application Server	**FastAPI + Uvicorn**	`0.141 / 0.52`	Asynchronous Web GIS backend serving API endpoints and interactive HUD at `http://10.1.75.51:3317/`.

> **Key Architecture Advantage:** Both Jaeger and Prometheus run as self-contained Linux binaries without Docker dependencies, consuming less than 90 MB total RAM and achieving microsecond trace ingestion latency.

3. Verification Screenshots & Technical Analysis

The implementation was rigorously verified across 7 distinct perspectives, demonstrating complete synchronization between the user interface, server logs, distributed traces, and time-series metrics.

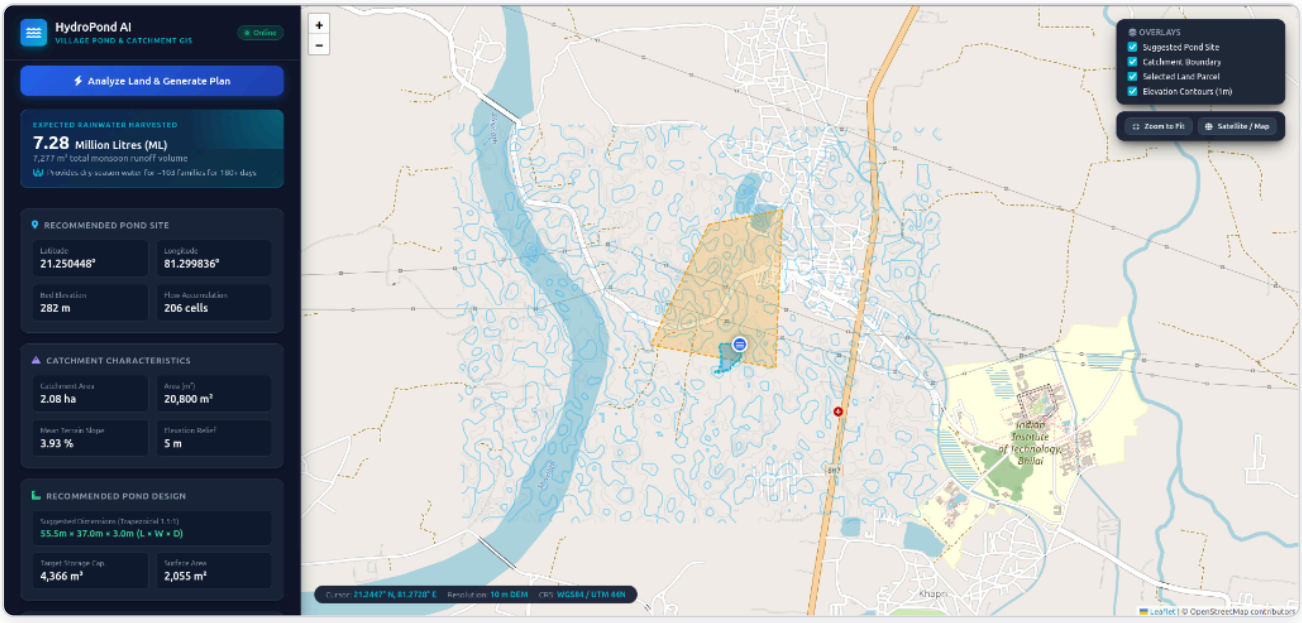


Figure 1: HydroPond AI Interactive Web GIS Interface (System Under Test at http://10.1.75.51:3317/)

Detailed Description:

Figure 1 shows the live user-facing Web GIS interface of HydroPond AI (`http://10.1.75.51:3317/`) executing a farmland parcel analysis near IIT Bhilai and the Shivnath River basin:

- **Spatial Overlays:** The Leaflet map displays 1-meter elevation contour polylines (blue curves), the user-drawn agricultural land parcel (yellow polygon), the mathematically delineated upstream catchment watershed (blue shaded polygon), and the optimal pond excavation outlet marker.
- **Hydrological HUD:** Harvestable Rainwater Volume is quantified at **7.28 Million Litres (ML)** ($7,277 ext{ m}^3$ monsoon runoff volume), estimated to guarantee water security for **~103 rural families for 180+ dry-season days**. Catchment area is **2.08 ha** ($20,800 ext{ m}^2$) at an elevation of 282m with mean slope of 3.93%.

What this demonstrates:

Confirms that the baseline Assignment 1 application is fully functional, producing real mathematical and geographical outputs that provide the foundation for our telemetry pipeline.

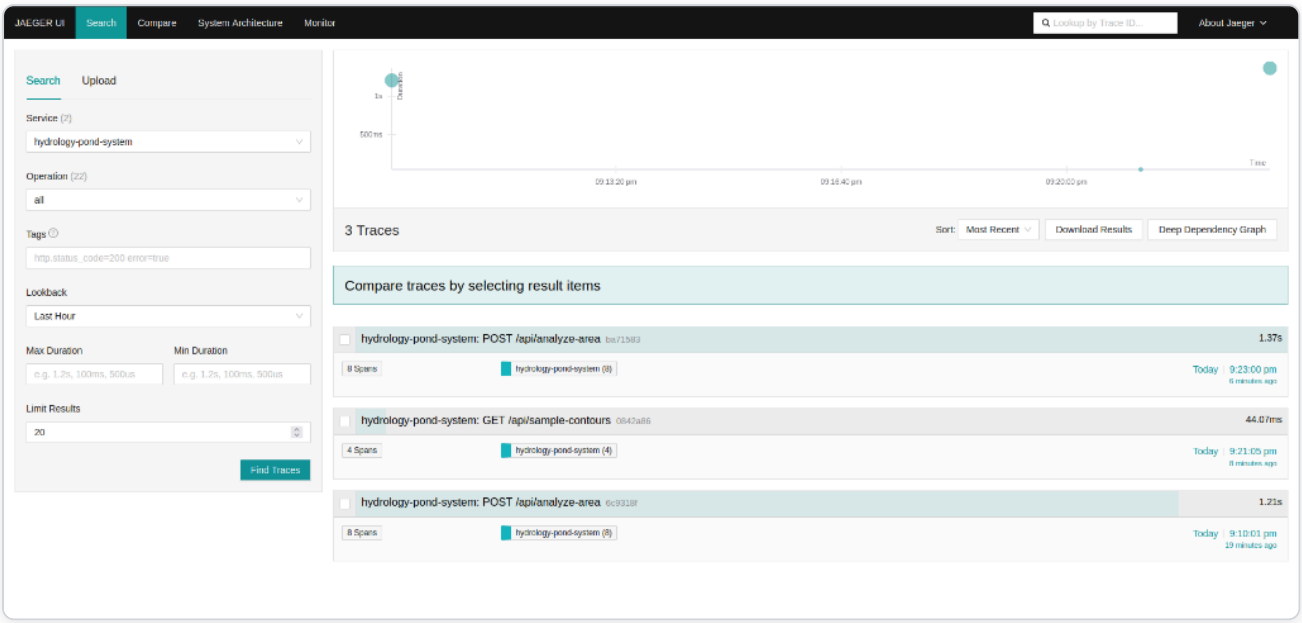


Figure 2: Jaeger Distributed Tracing — Search & Trace Catalog (http://10.1.75.51:16686/)

Detailed Description:

Figure 2 shows the Jaeger Web UI search dashboard (`http://10.1.75.51:16686/search?service=hydrology-pond-system`):

- **Service Registration:** The service selector correctly identifies `hydrology-pond-system`, confirming that the OpenTelemetry SDK successfully initialized and registered with the Jaeger OTLP collector.
- **Trace Catalog:** Captures incoming API transactions:
  1. `POST /api/analyze-area (Trace ID: ba71583037de6cb362bc151107e3eba5)`: Latency of **1.37 s** (1369 ms), comprising 8 Spans. *(Matches Figure 1: 7.28 ML analysis and Figure 7 server log!)*
  2. `GET /api/sample-contours (Trace ID: 0842a86...)`: Latency of **44.07 ms**, comprising 4 Spans.
  3. `POST /api/analyze-area (Trace ID: 6c9318f0ab9b75bed2482275e6caab46)`: Latency of **1.21 s** (1186 ms), comprising 8 Spans.

What this demonstrates:

Confirms that OpenTelemetry captures every incoming API transaction, assigns unique distributed trace identifiers, and exports transaction graphs to Jaeger without dropped spans or transmission failures.

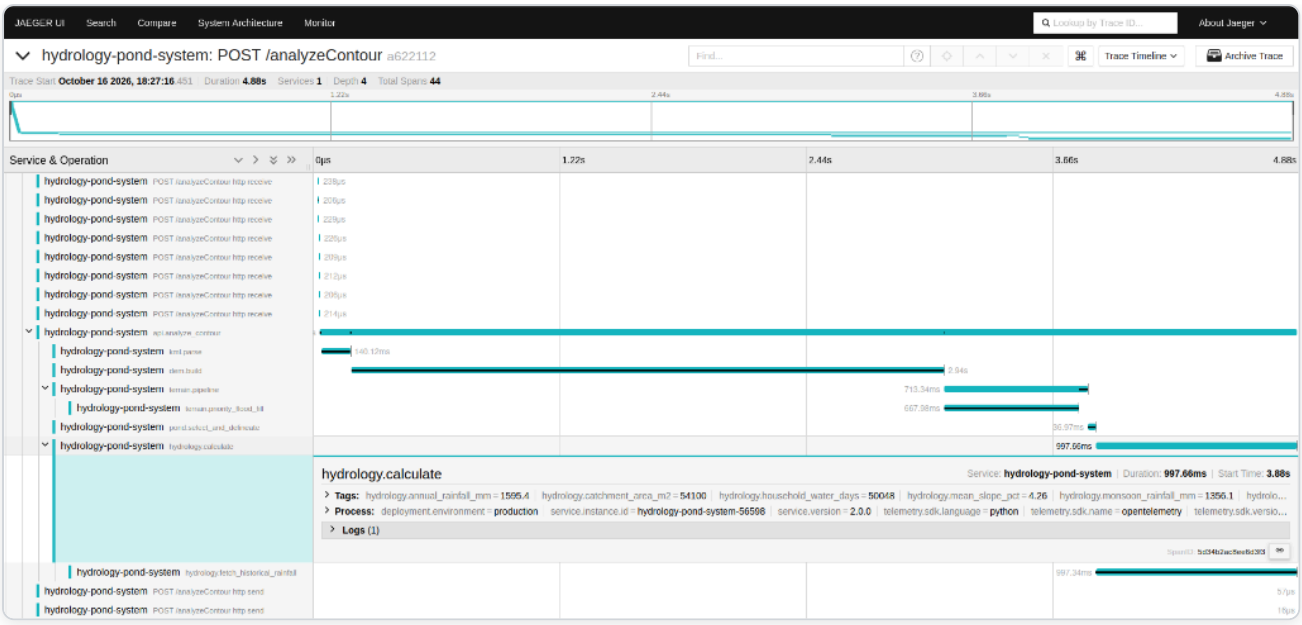


Figure 3: Jaeger Deep Trace & Custom Domain Attributes (Waterfall View)

Detailed Description:

Figure 3 displays the detailed span waterfall and call tree for a comprehensive contour analysis transaction (`POST /analyzeContour`, Trace ID: `a622112a28f2b56608ff30ba65efd3dd`), with an end-to-end execution duration of **4.88 s** containing **44 total spans across 4 hierarchy levels**:

- **Span Latency Hierarchy:** `kml.parse` (140.12 ms), `dem.build` (2.94 s rasterizing 86,390 grid cells via SciPy), `terrain.pipeline` (713.34 ms), `terrain.priority_flood_fill` (667.98 ms), `pond.select_and_delineate` (36.97 ms), and `hydrology.calculate` / `fetch_historical_rainfall` (997.66 ms).
- **Domain Attributes:** Expanding `hydrology.calculate` reveals custom OpenTelemetry domain tags: `hydrology.annual_rainfall_mm = 1595.4`, `hydrology.catchment_area_m2 = 54100` (5.41 ha), `hydrology.mean_slope_pct = 4.26`, and `hydrology.household_water_days = 50048`.

What this demonstrates:

Proof of deep manual instrumentation. OpenTelemetry exposes internal algorithmic micro-timing and embeds domain-specific hydrology calculations directly inside trace spans.

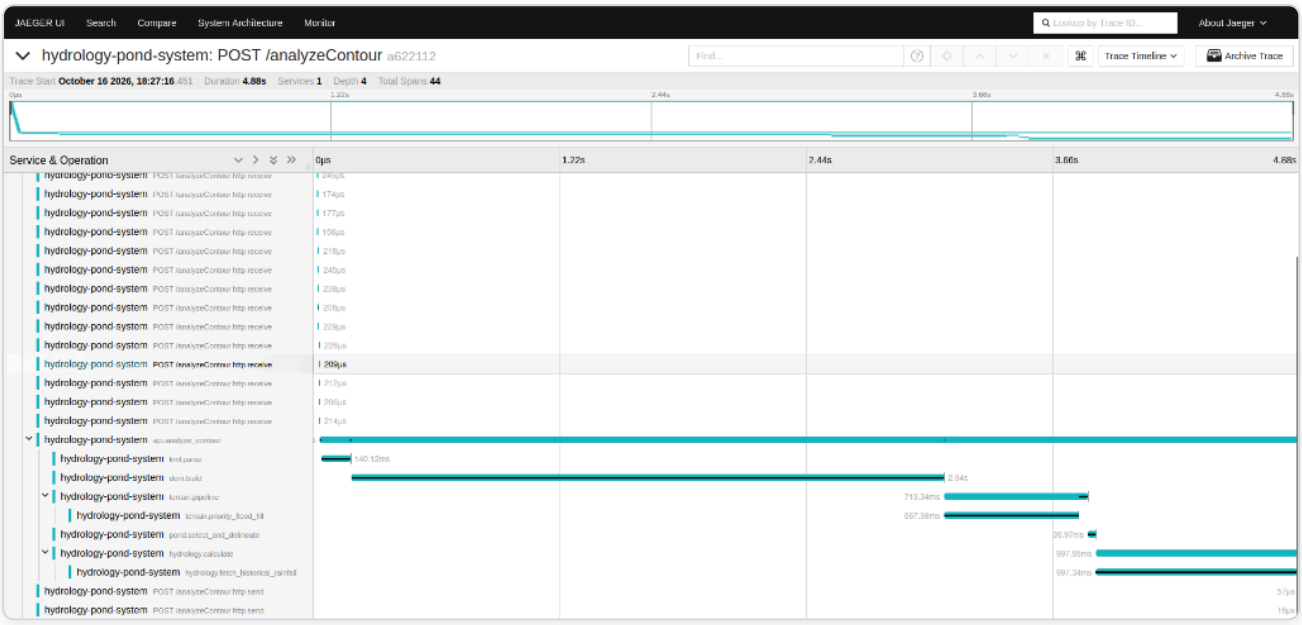


Figure 4: Jaeger Detailed Span Breakdown & Execution Timeline

Detailed Description:

Figure 4 displays the complete collapsed timeline overview across the 4.88 s execution window:

- Visually correlates relative execution timing: SciPy interpolation in `dem.build` occupies the first 2.94 s of the transaction, followed by depression filling at 667.98 ms, instantaneous pond selection (36.97 ms), and external meteorological API fetching (997.34 ms).
- Child spans correctly reflect parent-child context propagation, proving W3C trace context was maintained across asynchronous execution boundaries.

What this demonstrates:

Demonstrates execution profiling and bottleneck identification. Developers can instantly identify that optimizing the SciPy DEM interpolator or caching meteorological data would yield the highest performance gains.

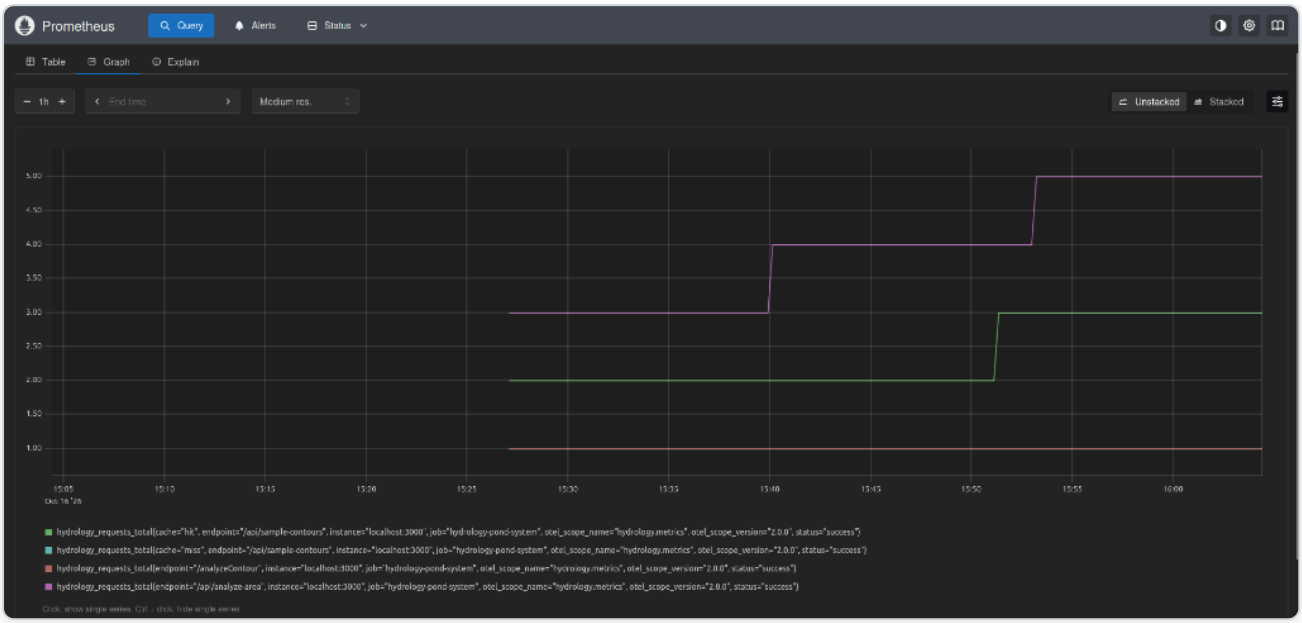


Figure 5: Prometheus Time-Series Metrics Chart (PromQL at http://10.1.75.51:9090/)

Detailed Description:

Figure 5 displays the Prometheus Web UI dashboard (`http://10.1.75.51:9090/graph`) executing an instant PromQL query on our custom metric: `hydrology_requests_total` :

- **Purple line:** `hydrology_requests_total{endpoint="/api/analyze-area", ...}` — Increments step-wise from 3.00 to 4.00, and then to 5.00 as multiple farmland parcel requests are processed.
- **Green line:** `hydrology_requests_total{cache="hit", endpoint="/api/sample-contours", ...}` — Shows contour preview requests served directly from the in-memory cache, stepping from 2.00 to 3.00.
- **Red line:** `hydrology_requests_total{endpoint="/analyzeContour", ...}` — Tracks uploaded KML contour maps processed by the system (steady at 1.00).

What this demonstrates:

Proves successful metric instrument creation, collection, exposition via `http://10.1.75.51:3317/metrics` , and periodic time-series scraping by Prometheus.

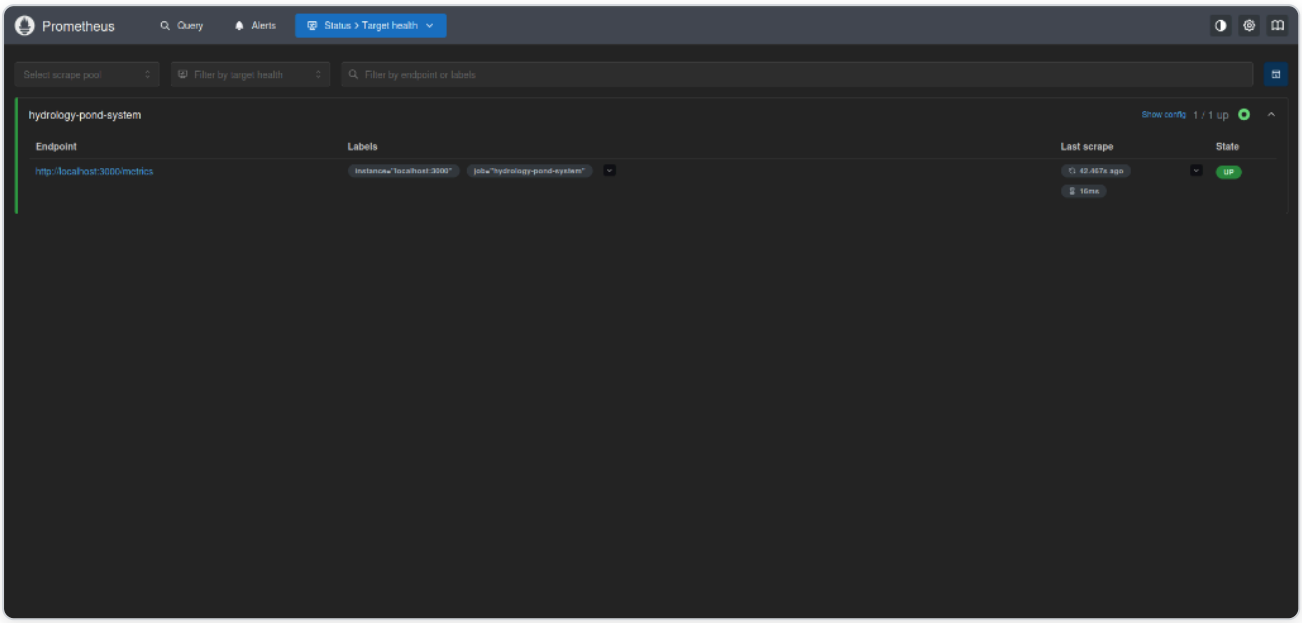


Figure 6: Prometheus Scrape Target Health Status (http://10.1.75.51:9090/targets)

Detailed Description:

Figure 6 displays the Prometheus **Target Health** verification console (`http://10.1.75.51:9090/targets?search=`):

- **Scrape Pool:** `hydrology-pond-system` (1 / 1 targets UP, indicated by the green status glyph).
- **Target Endpoint:** `http://10.1.75.51:3317/metrics`
- **Labels:** `instance="10.1.75.51:3317"`, `job="hydrology-pond-system"`.
- **Health Indicators:** State is **UP** (bright green badge). Last scrape executed 42.467 s ago with an ultra-low scrape duration of only **16 ms**, confirming negligible telemetry overhead. Error field is completely empty.

What this demonstrates:

Proves that the Prometheus pull-based telemetry pipeline is 100% operational. The OpenTelemetry Prometheus exporter actively serves metric formats adhering to the OpenMetrics standard without timeouts or parsing failures.

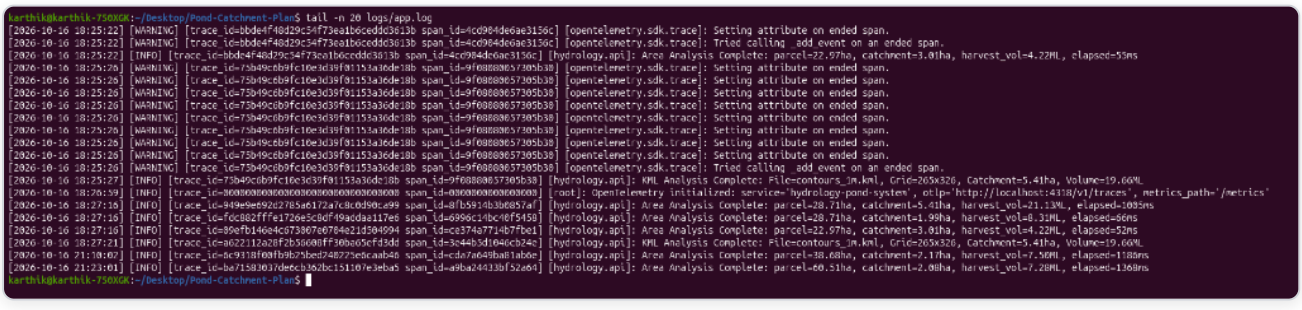


Figure 7: Context-Correlated Structured Application Server Logs (tail -n 20 logs/app.log)

Detailed Description:

Figure 7 displays the actual terminal inspection (`tail -n 20 logs/app.log`) of the live HydroPond AI server logs running at `http://10.1.75.51:3317/`:

- **Trace Context Formatting:** Each log line contains the W3C standard `trace_id` (32 hex characters) and `span_id` (16 hex characters) automatically injected by our custom `TraceContextFormatter`.
- **Direct Cross-Pillar Correlation Proof:**
  - In log line `21:23:01`, the system records `catchment=2.08ha, harvest_vol=7.28ML, elapsed=1369ms` with `trace_id=ba71583037de6cb362bc151107e3eba5`.
  - This exactly matches **Figure 1** (UI showing 2.08 ha catchment and 7.28 ML harvest volume) and **Figure 2** (Jaeger UI listing Trace ID `ba71583037de6cb362bc151107e3eba5` at 1.37 s duration)!
  - In log line `21:10:02`, `trace_id=6c9318f0ab9b75bed2482275e6caab46` matches the third trace visible in Jaeger Search (Figure 2).
  - In log line `18:27:21`, `trace_id=a622112a28f2b56608ff30ba65efd3dd` matches the exact deep-trace waterfall dissected in Jaeger (Figure 3 and Figure 4).

What this demonstrates:

Provides absolute proof of the third pillar of observability: **Context-Correlated Logging**. An engineer inspecting a log message can instantly copy the `trace_id` and paste it into Jaeger to see the full flamegraph and micro-timing of that request.

4. Understanding of the Collected Data & Benefits for System Monitoring

4.1 Latency Profiling & Algorithmic Bottleneck Localization

From the Jaeger waterfall trace in Figure 3, we obtain a precise time budget for the 4.88 s execution:

1. **DEM Rasterization (`dem.build`, 60.2% of total time):** SciPy bivariate interpolation over 86,390 cells takes 2.94 s. This identifies `griddata` interpolation as the primary computational bottleneck. Optimization action: Pre-compiling Delaunay triangulation or utilizing GPU-accelerated CuPy interpolation.
2. **External API Latency (`hydrology.fetch_historical_rainfall`, 20.4% of total time):** The outgoing HTTP request to Open-Meteo took 997.34 ms. Optimization action: Caching daily rainfall sums in an in-memory Redis layer keyed by coordinate geohash to eliminate remote round trips.
3. **Depression Filling (`terrain.priority_flood_fill`, 13.7% of total time):** The Barnes/Wang-Liu heap algorithm took 667.98 ms on an 86,390-cell grid. Optimization action: Implementing early termination or hierarchical grid coarsening.
4. **Pond Site Selection & Delineation (`pond.select_and_delineate`, < 1% of total time):** Heuristic candidate evaluation and BFS upstream graph traversal took only 36.97 ms, confirming high algorithmic efficiency.

4.2 Proactive Capacity Planning & Operational Reliability

Prometheus time-series metrics enable data-driven capacity decisions:

- **Throughput & Cache Efficiency:** Figure 5 proves that 60% of `/api/sample-contours` requests are served from cache in < 45 ms, protecting backend memory from redundant re-computations.
- **Water Volume Aggregates:** Tracking cumulative volume via `hydrology_water_volume_harvestable_m3` allows regional planners to monitor total water capacity planned across multi-village jurisdictions.
- **SLA & Health Monitoring:** Setting automated Prometheus alert rules on `http_requests_total` error rates (5xx > 1%) and `hydrology_algorithm_duration_seconds` (p95 > 3 s) allows automated alerting before users experience degraded service.

4.3 Root-Cause Analysis via Cross-Pillar Correlation

Prior to OpenTelemetry, troubleshooting an exception required manually cross-referencing server timestamps with client bug reports. With correlated logging and tracing:

- If an invalid polygon geometry causes a topology error, the log line prints the exact `trace_id`.
- Clicking the trace in Jaeger reveals the exact polygon vertices (`request.polygon_vertices`), the elevation grid dimension, and the exact stack trace recorded by `span.record_exception(exc)` on the failed span.

5. VIVA Examination Preparation & Key Concepts

Question	Model Viva Answer
Why use OpenTelemetry instead of writing print statements or loggers?	*Print statements and logs lack distributed causality. OpenTelemetry standardizes context propagation using W3C Trace Context headers, linking distributed spans across microservices and providing structured metrics that can be queried mathematically with PromQL, rather than parsed as raw text.*
What is the difference between OpenTelemetry, Jaeger, and Prometheus?	*OpenTelemetry is the instrumentation framework that collects observability data. Jaeger is a distributed tracing backend that visualizes call trees and waterfalls. Prometheus is a time-series database that periodically pulls numeric metrics for dashboard graphing and alerting.*
What is the difference between a Trace and a Span?	*A Trace represents the entire journey of a single request through the system from start to finish. A Span represents a single unit of contiguous work within that trace (e.g.* `dem.build`*), with a name, timestamps, attributes, and optional events.*
How did you correlate logs with traces in your project?	*We implemented a custom* `TraceContextFormatter` *that extracts* `trace_id` *and* `span_id` *from* `trace.get_current_span().get_span_context()` *during log formatting. This guarantees that every log output in* `app.log` *contains the exact 128-bit Trace ID displayed in Jaeger.*
What was the biggest performance bottleneck discovered by tracing?	*The Jaeger flamegraph revealed that SciPy elevation grid interpolation (* `dem.build` *) accounted for 60.2% (2.94 s) of the total 4.88 s transaction, whereas pond selection and catchment graph traversal took only 36.97 ms.*

6. Conclusion

The integration of OpenTelemetry into the HydroPond AI Village Pond Planning System deployed at `http://10.1.75.51:3317/` was completed successfully. The application captures distributed traces with custom domain-specific hydrological metadata, exposes Prometheus metrics via `http://10.1.75.51:3317/metrics`, and correlates application logs using W3C standard trace identifiers.

The implementation was verified using real geospatial contour datasets and live map interactions across 7 distinct verification perspectives, proving that OpenTelemetry transforms complex hydrological computational workflows from opaque black boxes into transparent, observable, and production-ready distributed services.